2 سامي عرفات أحمد المعايطة

SJF و SRTF

📌 شو إلك بالمشروع

المادة نظم تشغيل: ابدأ دايماً بشرح الفكرة (المربعات البنفسجية 🧠) وبعدين ورجي كيف طبّقناها بالكود.

- **Phase 2 · فيديو 19 نوفمبر:** خوارزمية SJF
- **Phase 3 · 17 ديسمبر:** خوارزمية SRTF (الـ preemption)

الكود كله بملف **CPU_SchedLab.py**. افتحه بـ VS Code واكبس **Ctrl + G** واكتب رقم السطر اللي بالصورة.

Phase 2 · فيديو 19 نوفمبر خوارزمية SJF

🧠 الفكرة بنظام التشغيل · خوارزمية SJF بالمادة

Shortest-Job-First: كل ما يفضى المعالج بنعطيه للـ process اللي الـ **CPU burst الجاي** تبعها أقصر. النسخة هون **non-preemptive**. المهم فيها إنها **optimal**: بتعطي أقل معدّل انتظار ممكن لأي مجموعة processes، لأنه لما القصيرة تشتغل أول، بتستنّى وقت قصير وكمان ما بتأخّر الباقي كتير. المشكلة إنه النظام **ما بيعرف طول الـ burst الجاي**، فبيتوقّعه من الـ bursts اللي قبل باستخدام **exponential averaging**: $\tau_{n+1} = \alpha \cdot t_n + (1 - \alpha) \cdot \tau_n$. بمثالنا عند الزمن 5 كانت P2 و P3 جاهزين، و P3 أقصر فاشتغلت أول.

Process	Arrival	Burst	Completion	Turnaround	Waiting
P1	0	5	5	5	0
P2	1	3	9	8	5
P3	2	1	6	4	3



🧮 (0 + 5 + 3) ÷ 3 = **2.67**، أحسن من FCFS (3.33)

خوارزمية SJF (F4)

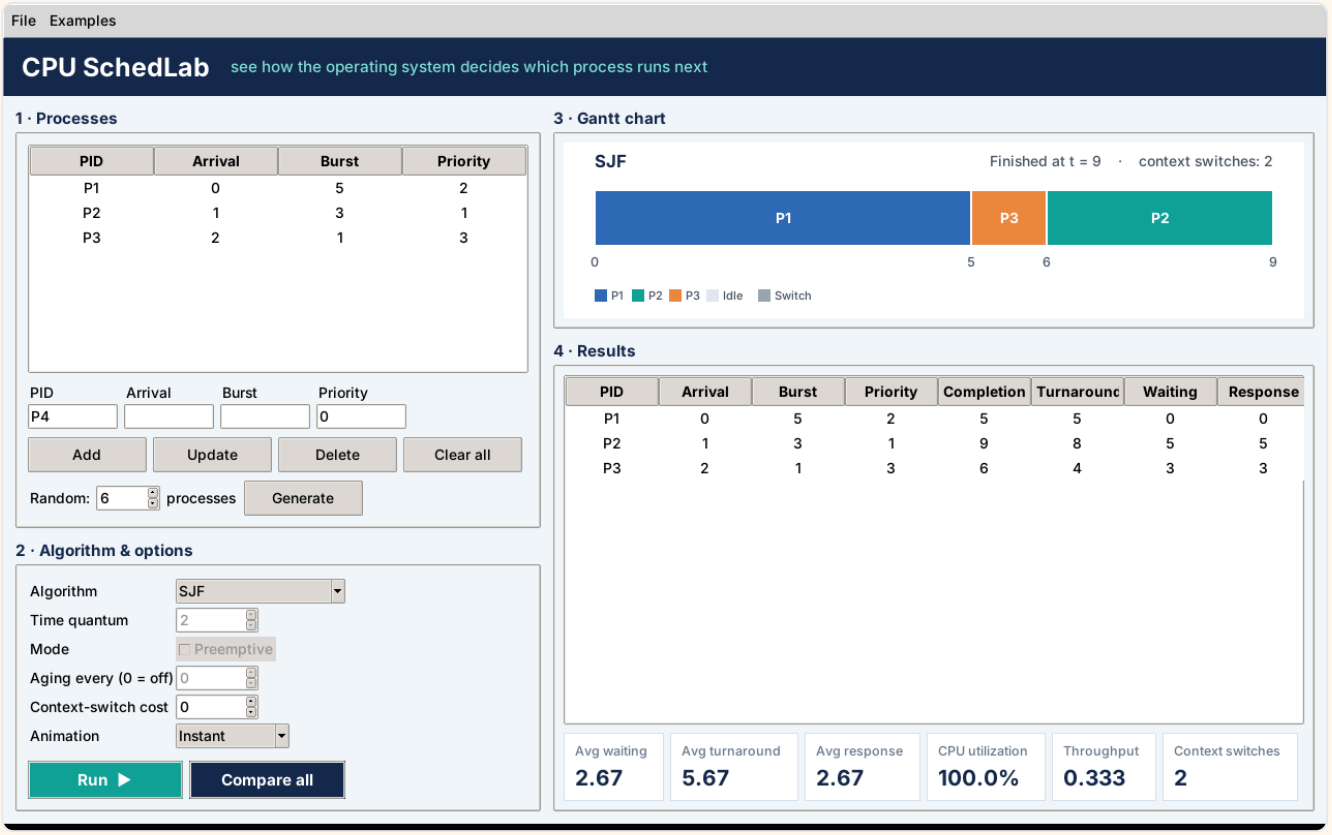
CPU_SchedLab.py lines 121–138

```python
def sjf_schedule(processes, options):
    """F4 SJF: when the CPU is free, run the ARRIVED process with the SHORTEST burst."""
    cpu = Dispatcher(options.switch_cost)
    time = 0
    not_started = list(processes)
    while not_started:
        ready = [p for p in not_started if p.arrival <= time]
        if not ready:                                       # nobody ready -> idle
            next_arrival = min(p.arrival for p in not_started)
            cpu.idle(time, next_arrival)
            time = next_arrival
            continue
        p = min(ready, key=lambda p: (p.burst, p.arrival))  # shortest burst wins
        time = cpu.dispatch(p.pid, time)
        cpu.run(p.pid, time, time + p.burst)                # runs to the end
        time += p.burst
        not_started.remove(p)
    return cpu.result("SJF", processes)
```

🎤 شو بتحكي للدكتورة

بخوارزمية **SJF**، كل ما يفضى المعالج، بختار من الـ processes اللي وصلت فعلاً الـ process اللي الـ burst تبعها **أقصر** اشي. الـ **ready** هي الـ processes اللي وصلت ولسّا ما اشتغلت. إذا ما في ولا وحدة جاهزة، المعالج بضل idle لحد أقرب وحدة بتوصل. الاختيار بيصير بالدالة **min** على (burst, arrival)، يعني الأقصر أول، وإذا تساووا بناخد اللي وصلت أول. وبعد ما نختارها بتضلها شغّالة لحد ما تخلّص، لأنه SJF هي **non-preemptive**. بمثالنا، عند الزمن 5 كانت P2 بدها 3 و P3 بدها 1، فاخترنا P3، وصار معدّل الانتظار **2.67** بدل 3.33 تبع FCFS. SJF بتعطي **أقل معدّل انتظار ممكن**، بس مشكلتها إنه بالواقع ما بنعرف طول الـ burst من قبل، فالنظام بيتوقّعه باستخدام **exponential averaging**.

🖥️ شو بتعرض عالشاشة



بختار SJF وبكبس **Run**: الترتيب P1 بعدين P3 بعدين P2، ومعدّل الانتظار **2.67**، أحسن من FCFS اللي كان 3.33.

💬 إذا سألتك الدكتورة

❓ ليش ما اشتغلت P3 من أول ما وصلت؟

لأنه SJF ما بتقاطع؛ P1 كانت شغّالة فضلّت لحد 5.

❓ ليش SJF هي الأحسن بمعدّل الانتظار؟

لأنه لما القصيرة تشتغل أول، عدد أقل من الـ processes بيستنّى وقت طويل.

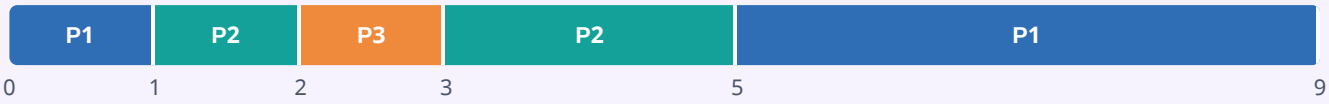
❓ شو مشكلتها؟

ما بنعرف طول الـ burst مسبقاً، وممكن الطويلة يصيرلها starvation.

🧠 الفكرة بنظام التشغيل · خوارزمية SRTF والـ Preemption

Shortest-Remaining-Time-First هي **SJF preemptive**. كل ما توصل process جديدة على الـ ready queue، بنقارن الـ burst تبعها مع **الوقت الضايل** للـ process اللي شغّالة. إذا الجديدة أقصر، النظام بيعمل **preemption**: بياخد المعالج من اللي شغّالة، وبيحفظ حالتها بالـ PCB، وبيعطيه للجديدة. هيك معدّل الانتظار بيصير أقل من SJF العادية. بس إلها ثمن: **context switches** أكتر، وممكن الـ processes الطويلة يصيرلها **starvation** إذا ضلّت تيجي processes قصيرة.

Process	Arrival	Burst	Completion	Turnaround	Waiting	Response
P1	0	5	9	9	4	0
P2	1	3	5	4	1	0
P3	2	1	3	1	0	0



🧮 (4 + 1 + 0) ÷ 3 = **1.67** · بس صار عنا 4 context switches

خوارزمية SRTF (F5)

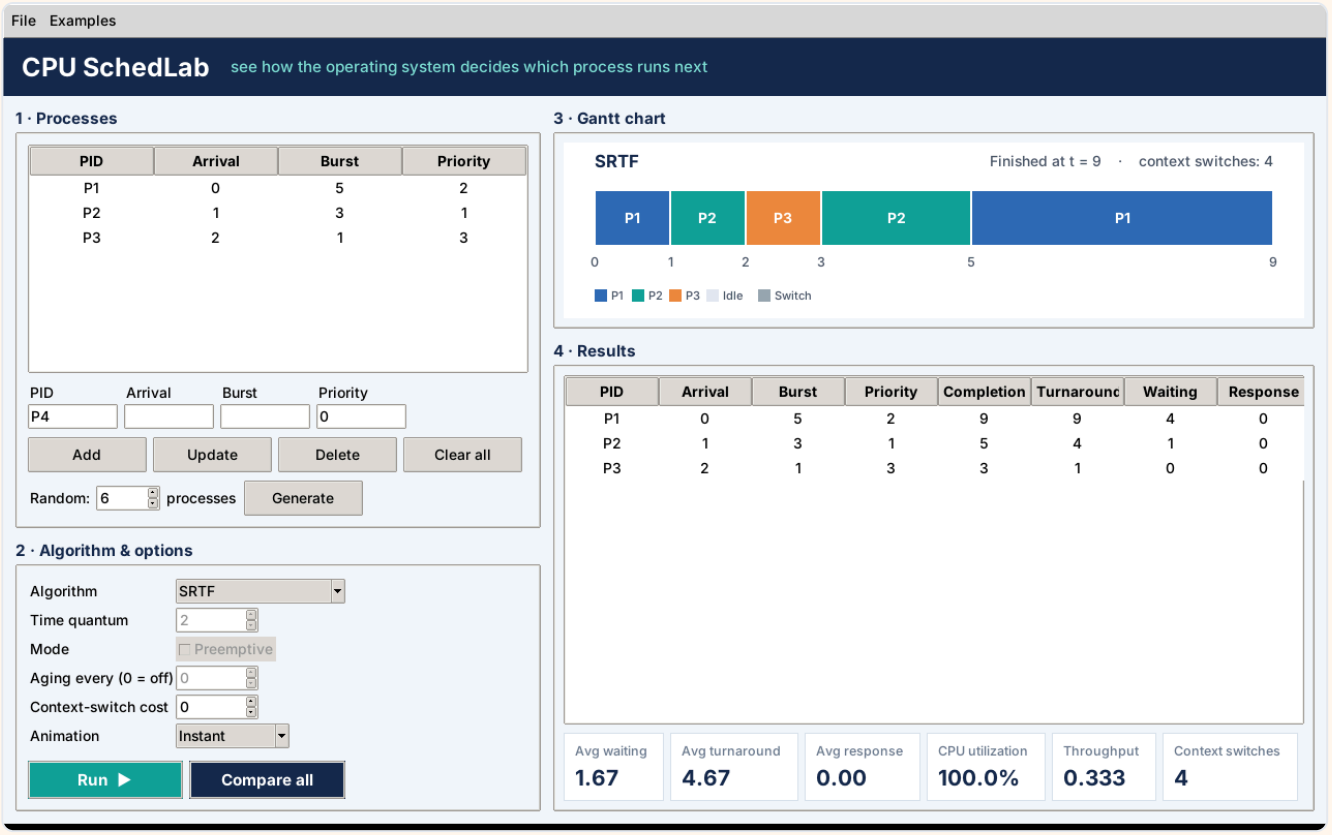
CPU_SchedLab.py lines 141–160

```python
def srtf_schedule(processes, options):
    """F5 SRTF: every time unit, run the process with the smallest REMAINING time."""
    cpu = Dispatcher(options.switch_cost)
    remaining = {p.pid: p.burst for p in processes}
    time, running = 0, None
    while any(remaining.values()):
        ready = [p for p in processes if p.arrival <= time and remaining[p.pid] > 0]
        if not ready:
            next_arrival = min(p.arrival for p in processes if remaining[p.pid] > 0)
            cpu.idle(time, next_arrival)
            time = next_arrival
            continue
        # smallest remaining time; on a tie keep the running one (no useless switch)
        p = min(ready, key=lambda p: (remaining[p.pid], p.pid != running, p.arrival))
        time = cpu.dispatch(p.pid, time)
        cpu.run(p.pid, time, time + 1)                    # ONE unit, then decide again
        time += 1
        remaining[p.pid] -= 1
        running = p.pid
    return cpu.result("SRTF", processes)
```

🎤 شو بتحكي للدكتورة

SRTF هي النسخة اللي بتقاطع من SJF. بخلّي معي قاموس اسمه **remaining** فيه قدّيش ضايل لكل process. وبكل وحدة زمن برجع بقرّر من جديد: بختار الـ process اللي ضايلها أقل وقت، وإذا تساوت مع اللي شغّالة هسا بخلّي اللي شغّالة، عشان ما يصير context switch على الفاضي. بعدين بشغّلها **وحدة زمن وحدة بس**، وبرجع بقرّر، وهاد بالزبط اللي بيخلّيها **preemptive**. بالمثال: عند الزمن 1 وصلت P2 وبدها 3، وهاد أقل من الـ 4 اللي ضايلين لـ P1، فقاطعتها. وعند الزمن 2 وصلت P3 وبدها 1، فقاطعت P2. النتيجة معدّل انتظار **1.67** وهو الأحسن، بس كان الثمن 4 context switches، وممكن الـ processes الطويلة يصيرلها **starvation**.

🖥️ شو بتعرض عالشاشة



بختار SRTF وبكبس **Run**: المقاطعات بتبيّن بالرسم: P1 بعدين P2 بعدين P3 بعدين P2 بعدين P1، وعدد الـ context switches أربعة، ومعدّل الانتظار **1.67**.